

Introduction to Machine Learning for Security

Instructor Guide — Storyboard, Teaching Notes, Demos & Tasks

Bachelor Business Informatics / Computer Science (5th–6th sem.)

90 minutes

Prof. Dr.-Ing. Sebastian Schlesinger, HWR Berlin

July 1, 2026

Didactic idea (Feynman principle). The audience already knows classical security cold (IDS, firewalls, YARA, crypto, cloud) but has *no* systematic ML background. Do *not* open with maths, datasets or algorithms. Tell a story: classical security worked for decades via rules and signatures; modern scale, novelty and evasion broke that model; ML is the *natural continuation*, not a new toy. Every new term is introduced only *after* the problem that motivates it. Target “aha” at the end: “*Now I understand why ML became necessary in security.*”

Contents

1	Didactic storyboard (the 90-minute arc)	1
2	Section-by-section teaching notes	2
2.1	Hook — security that no longer scales (slides 5–7)	2
2.2	Why classical methods hit a wall (slides 9–11)	3
2.3	Security is pattern recognition (slides 13–15) — THE PIVOT	3
2.4	Machine learning, intuitively (slides 17–20)	3
2.5	The three learning paradigms (slides 22–25)	4
2.6	Where ML lives today (slides 27–28)	4
2.7	What makes ML in security special (slides 30–32)	4
3	Three low-prep live demos	5
3.1	Demo 1 — “The room is the classifier” (no computer, 4 min)	5
3.2	Demo 2 — Learn a rule from 5 examples (Python, 6 min)	5
3.3	Demo 3 — The base-rate fallacy (Python or mental math, 4 min)	5
4	Three think-pair-share tasks	6
5	MCQ self-test (rationale)	7
6	The red-thread board (summary sketch)	7

1. Didactic storyboard (the 90-minute arc)

The lecture has four acts: *the problem* (why the old model breaks), *the idea* (what ML actually is), *the reality* (where it runs and why it is hard), and *the wrap-up*. Slide numbers refer to `ml_security_intro.pdf`.

Min	Segment	Slides	Mode / what happens
0–3	Title & framing	1–3	Read learning goals; promise “today is the <i>why</i> ”.

Min	Segment	Slides	Mode / what happens
3–15	Hook: security that no longer scales	5–7	<i>Interactive.</i> SOC night shift; the 450k/day number; three problems, one shape. Ask before revealing numbers.
15–27	Why classical methods hit a wall	9–11	Respect the toolbox (co-fill table); concrete YARA rule + “defeat it”; co-develop the five limits.
27–40	Security <i>is</i> pattern recognition	13–15	The pivot (<i>slow down</i>); reframe the whole curriculum; two worked examples → “read the rule off examples”.
40–55	Machine learning, intuitively	17–20	First say “ML”. Classical vs learned; build spam feature vector on whiteboard; draw the boundary; training vs inference.
55–65	The three learning paradigms	22–25	Supervised / unsupervised / reinforcement, each via a security example.
65–73	Where ML lives today	27–28	Map EDR/XDR/SIEM/UEBA to paradigms; foundation models & LLMs, hype-aware.
73–82	What makes ML special	30–32	The adaptive adversary; six hazards; adversarial ML (seed the rest of the series).
82–90	Wrap-up	34–39	Red-thread board; <i>one</i> think-pair-share; 3 MCQs; the one message.

Time-pressure plan. If you are behind at minute 60, drop to *one* worked example (slide 15), run *one* think-pair-share, and skip the MCQ (assign as homework). Never cut the hook or the pivot — they carry the whole argument.

2. Section-by-section teaching notes

2.1 Hook — security that no longer scales (slides 5–7)

Learning goals: students *feel* the scale/novelty problem before any ML vocabulary.

Core statements: (1) A SOC ingests 10^9 – 10^{10} events/day; one human reviews a few hundred. (2) $\approx 450,000$ new malware samples/day make one-signature-per-threat impossible. (3) Spam, malware and zero-days share one shape: *object* → *fast good/bad decision at unmanageable scale/variability*.

Visualisations & whiteboard: the SOC funnel (billions → 1 analyst); diverging curves (threat volume rises, hand-written signatures stay flat). Whiteboard: write “#events/day” and collect shouted estimates, then reveal.

Interactive questions: “How many events per day?” “Can we hand-write a signature for every new sample, forever?” The obvious “no” is the thesis of the lecture.

Short discussion: 60s — “what do you personally do when your inbox floods with spam you’ve never seen the exact wording of?”

Typical misconceptions: “AV already solved malware.” It solves the *known*; the problem is the daily flood of the *unknown*.

Transition to next: “Before we fix this, let’s respect the tools you already have — and find

exactly where they break.”

2.2 Why classical methods hit a wall (slides 9–11)

Learning goals: articulate *structural* (not accidental) limits of rules/signatures.

Core statements: signatures/rules/heuristics/firewalls/IDS/YARA all share one trait — a *human encodes “bad” in advance*. Five limits: scale, novelty (zero-day), evasion (polymorphism), combinatorics (bad = blend of many weak hints), maintenance (every rule is an FP risk and a forever cost).

Visualisations & whiteboard: fill the “method → how it decides → where you met it” table *with* the room; show the concrete YARA rule; “rules vs adaptive adversary” diagram.

Interactive questions: on the YARA slide — “You are the attacker: name one trivial change that defeats this exact-match rule.” (rename URL, XOR strings, reorder opcodes.)

Short discussion: “Why does adding *more* rules not solve it?” (more FPs, more maintenance, still zero coverage of the genuinely new).

Typical misconceptions: “*Just write better/more rules.*” Confuses effort with generalisation — rules only ever encode the past.

Transition to next: “So we can’t enumerate ‘bad’. But look what all these tasks actually are...”

2.3 Security is pattern recognition (slides 13–15) — THE PIVOT

Learning goals: recognise that security tasks are *classification* or *anomaly detection* problems.

Core statements: object → decision → action. Classification = “which known category?”; anomaly detection = “does this deviate from normal?” (for when you can’t enumerate bad, only usual — ideal for zero-days). Signatures were always doing classification, with a hand-written rulebook.

Visualisations & whiteboard: object → decision → action flow; the 9-row curriculum table (each: object / decision / why rules get hard); two worked examples (DNS tunnelling, UEBA).

Interactive questions: “Give me the *object* and the *decision* for phishing detection.” Cold-call two students; let them populate a table row live.

Short discussion: “For UEBA, why is there no single global ‘normal’?” (normal is *per person*: 2 a.m. login is fine for an admin, alarming for HR).

Typical misconceptions: “*Anomaly = attack.*” No — anomaly = *unusual*, which is a *superset* of attacks (also: new intern, marketing campaign). That gap is the false-positive problem, foreshadowing Part III.

Transition to next: “We have piles of examples but can’t write the rule. What if the machine read the rule off the examples itself?” — hand-off into ML.

2.4 Machine learning, intuitively (slides 17–20)

Learning goals: define data/features/labels/training/inference/generalisation without maths.

Core statements: ML = *learn* the decision rule from examples instead of writing it. Features are the numeric clues per object (feature engineering *is* the security expertise). Training = fit a boundary to labelled examples; inference = which side is a new object on; generalisation = works on unseen objects, does not just memorise.

Visualisations & whiteboard: classical (humans write rules) vs ML (data+answers → learned rule); the spam feature-vector table; the 2-D scatter with a learned boundary and one “new?” point; training-vs-inference pipeline (two clocks).

Visualisations & whiteboard (whiteboard): build the spam feature vector live — ask the room “what would you *measure* about an email to guess spam?” and write each suggestion as a column (#links, %CAPS, has “invoice”...). This makes “features” their idea, not yours.

Interactive questions: “The orange dot is a brand-new email we’ve never seen. Which side of the boundary — and what does the model predict?”

Typical misconceptions: “*The model memorises all the malware.*” It stores a *boundary*, not the samples — that is why it catches unseen variants. Also “*the AI keeps learning live from every attack*” — usually it trains periodically and only *infers* live.

Transition to next: “There’s more than one way to learn — and each maps to a security job.”

2.5 The three learning paradigms (slides 22–25)

Learning goals: distinguish supervised / unsupervised / reinforcement by their security use.

Core statements: supervised = labelled examples (spam/malware/phishing classifiers); unsupervised = no labels, find “normal” and flag deviations (UEBA, network anomaly), or cluster malware families; reinforcement = act, get reward, adapt (auto-response, adaptive honeypots, fuzzing agents).

Visualisations & whiteboard: the three-box triad + the “you give it.../ security example” table; the anomaly ellipse with an outlier; the agent↔environment loop.

Interactive questions: “You must catch *insider* data theft with almost no labelled insider examples. Which paradigm?” (unsupervised — see TPS #2).

Short discussion: “Why is supervised learning ‘only as good as its labels’, and why are labels so hard in security?” (attackers hide ⇒ ‘benign’ may be ‘not caught yet’).

Typical misconceptions: “*More data always = better model.*” Mislabelled/biased data makes it confidently wrong.

Transition to next: “You now hold the three keys. Watch how every security product is just these three, pointed at a different object.”

2.6 Where ML lives today (slides 27–28)

Learning goals: map buzzwords (EDR/XDR/SIEM/UEBA) to paradigms; place LLMs soberly.

Core statements: every product category is one of three paradigms on a different object. Foundation models/LLMs are supervised learning at huge scale: useful for alert-summarisation and SOC copilots (defence) and for scaled phishing / prompt injection (offence). Principles *and* pitfalls from today still apply — amplified.

Visualisations & whiteboard: the “where / what the ML does / which paradigm” table.

Interactive questions: “Your EDR flags a never-before-seen process. Supervised or anomaly — and why probably *both*?”

Typical misconceptions: “*AI will replace the SOC analyst.*” Today it *augments*: it drafts, ranks, explains — a human still decides and is accountable.

Transition to next: “If it’s this useful, why isn’t security solved? Because security ML is unlike any other ML...”

2.7 What makes ML in security special (slides 30–32)

Learning goals: explain why an adaptive adversary changes everything; name the six hazards and adversarial ML; leave students curious for the series.

Core statements: in normal ML the world is indifferent (cats don’t restyle to fool you); in security the data is made by an adversary trying to defeat your model. Six hazards: adaptive adversary, data/concept drift, class imbalance & base rate, FP/FN asymmetry, explainability, adversarial ML. Adversarial ML = evasion (fool what it sees) and poisoning (corrupt what it learns).

Visualisations & whiteboard: cat-vs-malware contrast; the six-hazard two-column list; the adversary→{poison training, evade inference} diagram.

Visualisations & whiteboard (whiteboard, 60s): the base-rate fallacy — 1 attack per 1,000,000 events, 99.9% accuracy ⇒ ~1,000 false alarms for the 1 real attack. This is the single most important number in the lecture (see Demo 3).

Interactive questions: “Why is ‘99% accuracy’ a meaningless headline for a rare-attack detector?”

Typical misconceptions: “*99% accuracy = great detector.*” With extreme imbalance, accuracy is

dominated by the majority class — ask for false-positive rate and recall instead.

Transition to next: “Each of these hazards is a future lecture. Let’s tie the whole hour together.”

3. Three low-prep live demos

All three run in the lecture hall with nothing but a browser (or a projector and a whiteboard). Each has an offline fallback so it never fails live.

3.1 Demo 1 — “The room is the classifier” (no computer, 4 min)

Setup: show 5–6 one-line emails on a slide, e.g. “*URGENT: your invoice is overdue, click <http://bit.ly/x> to pay*”, “*Lunch at 12:30?*”, “*FREE!!! CLAIM YOUR PRIZE NOW*”, etc.

Run: for each email, the room votes spam/ham by show of hands. Tally.

Reveal: ask “*what made you vote spam?*” Write their answers as columns: #links, ALL-CAPS, urgency words, mangled spelling, unknown sender.

Punchline: “You just did *feature engineering* and ran a *classifier* in your heads. A model does exactly this — it just learns how much to weigh each clue, from thousands of examples.”

This demo motivates slides 18–19 perfectly; run it immediately before them.

3.2 Demo 2 — Learn a rule from 5 examples (Python, 6 min)

Paste into a Google Colab / Jupyter cell (or run locally). It trains a tiny model on the exact feature table from slide 18 and predicts a new email.

```
from sklearn.tree import DecisionTreeClassifier, export_text

# features: [num_links, percent_caps, has_"invoice"] (1 = yes)
X = [[9,40,1],[1,5,0],[12,60,1],[0,8,0],[7,33,1],[2,10,0]]
y = ["spam","ham","spam","ham","spam","ham"]

clf = DecisionTreeClassifier(max_depth=2).fit(X, y)

# a brand-new email the model has never seen:
new = [[8, 35, 1]]
print("prediction:", clf.predict(new)[0])
print(export_text(clf, feature_names=["links","caps","invoice"]))
```

Expected output (offline fallback — read it out if no internet):

```
prediction: spam
|--- caps <= 21.50
|   |--- class: ham
|--- caps > 21.50
|   |--- class: spam
```

Talking point: the machine *wrote its own rule* (“if %CAPS > 21.5 then spam”). Nobody hand-coded that threshold — it was *learned* from the examples. That is the whole idea of the lecture, in five lines.

3.3 Demo 3 — The base-rate fallacy (Python or mental math, 4 min)

Run at the “six hazards” slide (31). Even a near-perfect detector drowns analysts when attacks are rare.

```

events      = 1_000_000      # events in a day
attacks     = 1              # true attacks among them
accuracy    = 0.999          # "99.9% accurate" detector
fp_rate     = 1 - accuracy    # 0.1% of benign events misflagged

false_alarms = round((events - attacks) * fp_rate)
print(f"real attacks:  {attacks}")
print(f"false alarms:  {false_alarms}")
print(f"an analyst chasing an alert is right ~{attacks/false_alarms:.2%}
      of the time")

```

Expected output:

```

real attacks:  1
false alarms:  999
an analyst chasing an alert is right ~0.10% of the time

```

Punchline: “99.9% accuracy” produces ~1000 false alarms for every real attack. *This* is why raw accuracy is the wrong metric and why alert fatigue is a security-ML problem, not a laziness problem.

4. Three think-pair-share tasks

Format each: *think* alone 1 min → *pair* 2 min → *share* 1 min. Run one live (slide 35); the others are backup or homework.

TPS 1 — Rule vs. model (robustness)

Task. For *DNS tunnelling*: (a) write the `if...then` rule you would deploy; (b) list the *features* you would hand a classifier. Which is more robust to an adversary, and why?

Expected direction. The rule (e.g. “block queries >52 chars”) is trivially evaded by staying just under the threshold; the classifier weighs many soft features (length *and* entropy *and* query rate *and* rare subdomains) so no single tweak defeats it — but it costs interpretability and can false-positive on legit CDNs. Good answers name the trade-off, not a winner.

TPS 2 — Choose the paradigm

Task. Detect *insider* data theft with almost no labelled examples of insiders. Supervised, unsupervised, or reinforcement — one sentence why?

Expected answer. Unsupervised / anomaly detection: with (almost) no labels you cannot train a supervised classifier, so you model each user’s *normal* behaviour and flag deviations (UEBA). Bonus insight: you’ll still face many false positives (normal changes), which is the base-rate problem again.

TPS 3 — Cost of errors sets the threshold

Task. Airport malware scanner vs. consumer email spam filter: which error is worse, a false positive or a false negative? Set the threshold.

Expected answer. Airport: a false *negative* (missed malware) is catastrophic ⇒ lower the threshold, accept more false positives / manual review. Spam filter: a false *positive* (a real invoice

in junk) is costly to the user $\Rightarrow$ raise the threshold, tolerate some spam. Point: the threshold is a *business/risk* decision, not a maths constant.

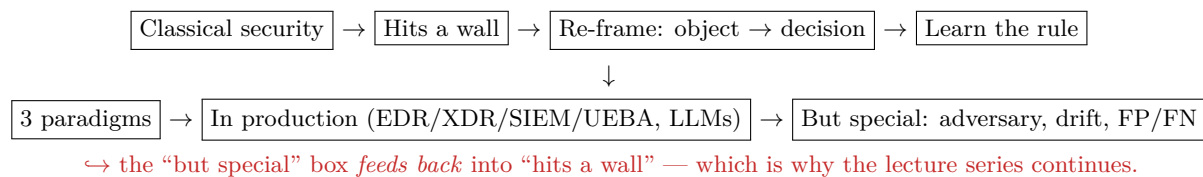
5. MCQ self-test (rationale)

The three questions appear on deck slide 36; answers on slide 37. Rationale for the instructor:

Q	Key	Why (and the tempting distractor)
Q1	B	A signature encodes the <i>past</i> ; a zero-day is new, so nothing matches. Distractor A (“encrypted”) tempts students who conflate evasion mechanisms.
Q2	B	No labels $\Rightarrow$ model “normal” and flag deviations = anomaly detection. Distractor A (supervised) fails because there is nothing to label.
Q3	B	Base-rate fallacy: at 1-in-a-million, 0.1% error = 1,000 false positives per real attack. This is the concept from Demo 3; if the room misses it, re-run the calculation.

6. The red-thread board (summary sketch)

Reproduce this on the whiteboard as the lecture proceeds (one box per act); reveal the complete version on deck slide 34. Read strictly left to right.



Narration for the board. “The *problem* forced the *re-frame*; the re-frame invited *learning*; learning brought *new problems* — and those new problems are the rest of this course.”

The one message (closing, deck slide 39). *Machine learning does not replace classical security — it extends it everywhere explicit rules are no longer enough.* Signatures for the known; learning for the new, the vast, and the changing. Together.